



Improving PostgreSQL Join Estimates - Statistics and Beyond

Alexandra Wang, EDB
Percona Live 2026, Mountain View



Agenda

- The Problem
- A Practical Fix: Join Statistics
 - Syntax & Key Idea
 - Performance Results
- Alternative and Open Questions
- Beyond Join Statistics

The Achilles' Heel of Cost-Based Optimization

“The root of all evil,
the Achilles Heel of query optimization,
is the estimation of the size of intermediate results,
known as cardinalities.”

“...the cost model may introduce errors of at most 30% for a given cardinality,
but the cardinality model can quite easily introduce errors of many **orders of magnitude!**”

– Guy Lohman, IBM Research

"Is Query Optimization a 'Solved' Problem?" (2014)

The Independence Assumption - Multiple Columns

$P(A \text{ AND } B) = P(A) \times P(B)$ The planner assumes independence until told otherwise.

*SELECT * FROM customers WHERE city = 'Mountain View' AND state = 'California';*

$P(\text{city} = \text{'Mountain View'}) = 0.056$ ← from pg_stats MCV

$P(\text{state} = \text{'California'}) = 0.05$ ← from pg_stats MCV

$P(\text{Mountain View AND California}) = 0.056 \times 0.05 = 0.0028$

Estimated rows: $0.0028 \times 200,000 = 560$

Actual rows: 5,000 ← **9x off**

The majority of the Mountain View rows are in California — they are functionally dependent.

** There are “Mountain View” cities in other states but much less population*

Extended Statistics

Defines data correlation between multiple columns

Example:

```
-- Tells planner: city and state are correlated  
CREATE STATISTICS s1 (ndistinct, mcv) ON city, state FROM customers;
```

- Introduced PostgreSQL 10 (Tomas Vondra, 2017)
- Captures correlations **WITHIN a single table**
- Stat kinds: **ndistinct, functional dependencies, MCV lists**

Extended Statistics - Current Limits

Single-table only. Cannot help with joins.

What happens when the correlation **crosses a join boundary**?

```
SELECT *  
FROM A JOIN B  
ON (B.id = A.foreign_key)  
WHERE B.filter_col = 'popular-filter-value';
```

- Filter on dimension table (B)
- Join key correlation in fact table (A)
- Stats stored per relation
- Cross-table correlation invisible

The Join Order Benchmark (JOB)

- Uses the IMDB data set, 113 queries
- Queries have between 3 and 16 joins, with an average of 8 joins per query

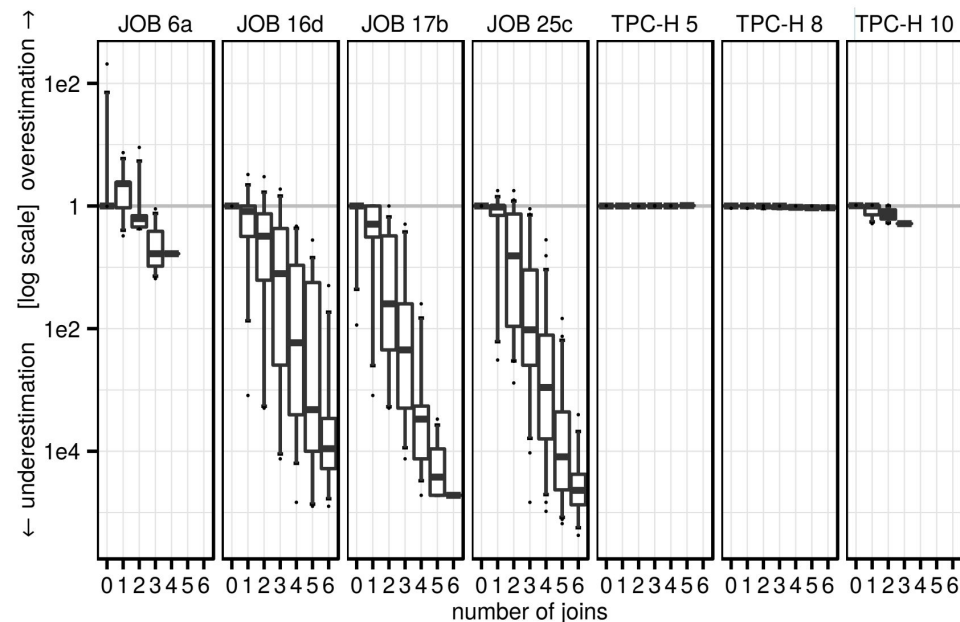


Figure 4: PostgreSQL cardinality estimates for 4 JOB queries and 3 TPC-H queries

How Good Are Query Optimizers, Really?

Viktor Leis
TUM
leis@in.tum.de
Peter Boncz
CWI
p.boncz@cwi.nl

Andrey Gubichev
TUM
gubichev@in.tum.de
Alfons Kemper
TUM
kemper@in.tum.de

Atanas Mirchev
TUM
mirchev@in.tum.de
Thomas Neumann
TUM
neumann@in.tum.de

ABSTRACT

Finding a good join order is crucial for query performance. In this paper, we introduce the Join Order Benchmark (JOB) and experimentally revisit the main components in the classic query optimizer architecture using a complex, real-world data set and realistic multi-join queries. We investigate the quality of industrial-strength cardinality estimators and find that all estimators routinely produce large errors. We further show that while estimates are essential for finding a good join order, query performance is unsatisfactory if the query engine relies too heavily on these estimates. Using another set of experiments that measure the impact of the cost model, we find that it has much less influence on query performance than cardinality estimates. Finally, we investigate plan enumeration techniques comparing exhaustive dynamic programming with heuristic algorithms and find that exhaustive enumeration improves performance despite the sub-optimal cardinality estimates.

INTRODUCTION

One of the most studied problems in the database field is the problem of finding a good join order. Figure 1 illustrates the classical, ad-hoc approach, which dates back to System R [36]. To obtain a query plan, the query optimizer enumerates some subset of valid join orders, for example using dynamic programming. Cardinality estimates as its principal input, the cost model chooses the cheapest alternative from semantically equivalent alternatives. If the cardinality estimates and the cost model are accurate, this architecture obtains the optimal query plan. In reality, cardinality estimates are usually computed based on simple assumptions like uniformity and independence. In real data sets, these assumptions are frequently wrong, which leads to sub-optimal and sometimes disastrous plans. In our experiments and analyses paper we investigate the three components of the classical query optimization architecture to answer the following questions:

- How good are cardinality estimators and when do bad estimates lead to slow queries?

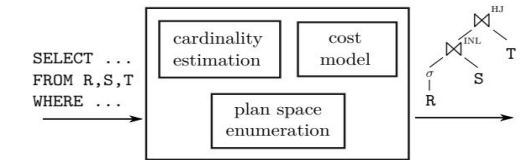


Figure 1: Traditional query optimizer architecture

- How important is an accurate cost model for the overall query optimization process?
- How large does the enumerated plan space need to be?

To answer these questions, we use a novel methodology that allows us to isolate the influence of the individual optimizer components on query performance. Our experiments are conducted using a real-world data set and 113 multi-join queries that provide a challenging, diverse, and realistic workload. Another novel aspect of this paper is that it focuses on the increasingly common main-memory scenario, where all data fits into RAM.

The main contributions of this paper are listed in the following:

- We design a challenging workload named *Join Order Benchmark (JOB)*, which is based on the IMDB data set. The benchmark is publicly available to facilitate further research.
- To the best of our knowledge, this paper presents the first end-to-end study of the join ordering problem using a real-world data set and realistic queries.
- By quantifying the contributions of cardinality estimation, the cost model, and the plan enumeration algorithm on query performance, we provide guidelines for the complete design of a query optimizer. We also show that many disastrous plans can easily be avoided.

The rest of this paper is organized as follows: We first discuss important background and our new benchmark in Section 2. Section 3 shows that the cardinality estimators of the major relational database systems produce bad estimates for many realistic queries, in particular for multi-join queries. The conditions under which these bad estimates cause slow performance are analyzed in Section 4. We show that it very much depends on how much the query engine relies on these estimates and on how complex the physical database design is, i.e., the number of indexes available. Query engines that mainly rely on hash joins and full table scans,

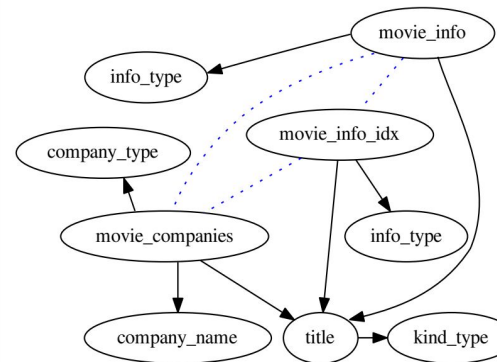


Figure 2: Typical query graph of our workload

The Join Order Benchmark (JOB) - PostgreSQL in 2026

Setup:

- On my Macbook: `./configure 'CFLAGS=-g -O3'`
- Cold runs: restarted database & cleared system cache
- Warm runs: repeat run of the entire test suite
- Average of 3 runs per test

JOB Results - Total Execution Time (pg19devel)

Configuration	Cold Runs (s)	Warm Runs (s)
Default settings	140.75 ± 2.1	117.72 ± 0.8
SET enable_nestloop = off	87.16 ± 0.2	86.68 ± 0.4

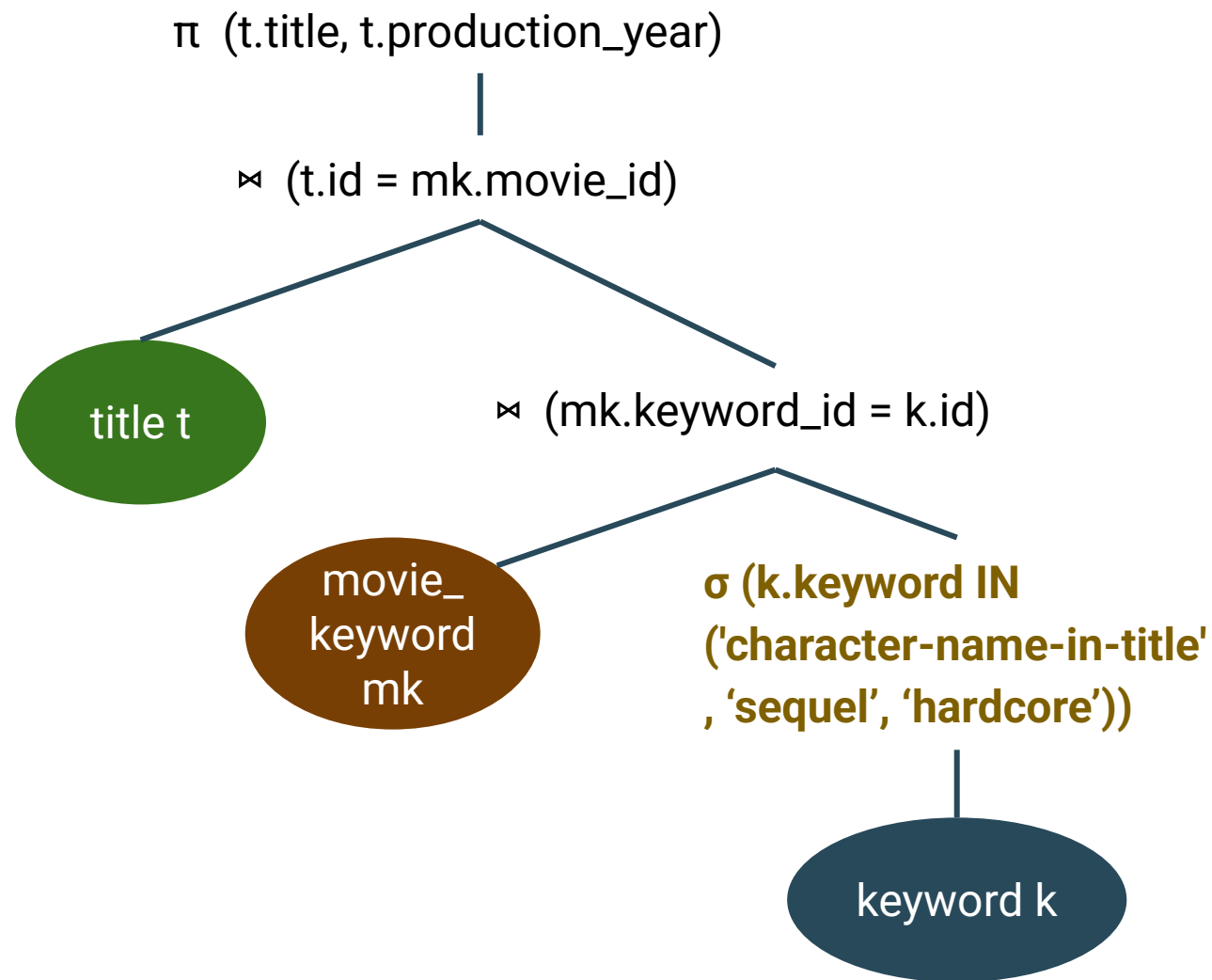
When Nest Loop Joins are disabled:

Cold run: **38% faster than default**

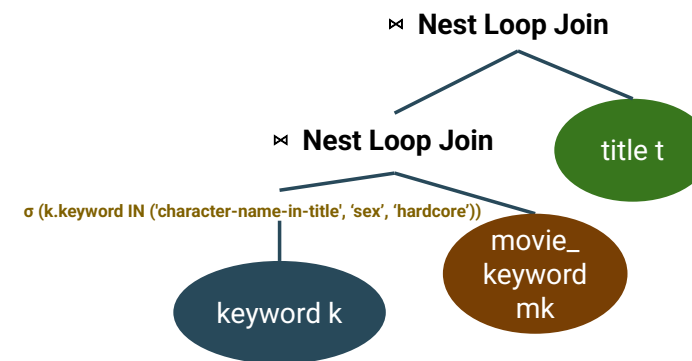
Warm run: **26% faster than default**

A Minimal Example

```
EXPLAIN (ANALYZE)
SELECT t.title, t.production_year
FROM keyword k
JOIN movie_keyword mk ON mk.keyword_id = k.id
JOIN title t ON t.id = mk.movie_id
WHERE k.keyword IN ('character-name-in-title',
                   'sequel',
                   'hardcore');
```



EXPLAIN ANALYZE - Default Setting



Nested Loop (rows=101) (actual ... rows=172784.00 loops=1)

-> **Nested Loop (rows=101) (actual ... rows=172784.00 loops=1)**

-> **Seq Scan on keyword k (rows=3) (actual ... rows=3)**

Filter: (keyword = ANY ('{character-name-in-title,sex,hardcore}'::text[]))

-> **Bitmap Heap Scan on movie_keyword mk (rows=307) (actual ... rows=57594.67 loops=3)**

-> Bitmap Index Scan on keyword_id_movie_keyword

Index Cond: (keyword_id = k.id)

-> **Index Scan using title_pkey on title t (rows=1) (actual ... rows=1.00 loops=172784)**

Index Cond: (id = mk.movie_id)

Planning Time: 11.314 ms

Execution Time: 1340.905 ms

EXPLAIN ANALYZE - Set enable_nestloop = off

Hash Join (rows=101) (actual rows=172784.00)

Hash Cond: (t.id = mk.movie_id)

-> **Seq Scan** on title t

-> **Hash** (rows=101) (actual rows=172784.00)

-> **Hash Join** (**rows=101**) (**actual rows=172784.00**)

Hash Cond: (mk.keyword_id = k.id)

-> **Seq Scan** on movie_keyword mk

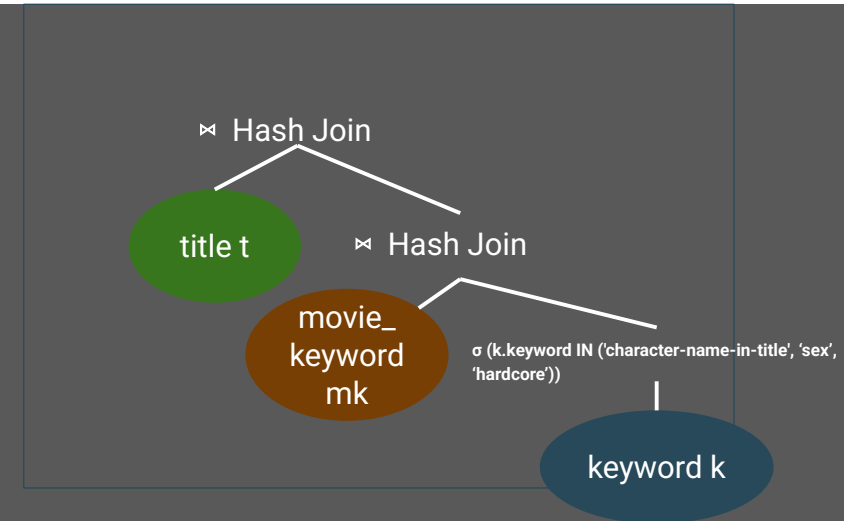
-> **Hash**

-> Seq Scan on keyword k (rows=3) (actual rows=3.00 loops=1)

Filter: (keyword = ANY ('{character-name-in-title,sex,hardcore}'::text[]))

Planning Time: 0.435 ms

Execution Time: 581.273 ms



**Omitted irrelevant details*

**Also SET max_parallel_workers_per_gather = 0; to make plan more readable and easier to compare*

Execution Time

	Planner's Estimate	Reality
Rows	307	172784
Plan Chosen	2 Nest Loop Joins	2 Hash Joins
Executions Time	1,341 ms	581 ms (252 ms when parallel)

A Practical Fix: Join Statistics

Join Statistics

Goal:

- Capture **join correlations** between tables
- Start with Join **MCV**, extend later to **NDistinct / dependencies**
- **Low overhead** during ANALYZE
- Simple, **extensible design**

Not Goal:

- Automatically detect candidate columns
- Automatically create the join statistics

Join Statistics - Proposed Syntax for MCV Stats

```
CREATE STATISTICS [ [ IF NOT EXISTS ] statistics_name ]  
    [ ( mcv ) ]  
    ON { table_name1.column_name1 [, { table_name1.column_name2 } [, ...]]  
    FROM table_name1 JOIN table_name2 ON table_name1.column_name3 = table_name2.column_name4
```

Example:

```
-- Create cross-table MCV statistics on a single filter column (keyword)  
CREATE STATISTICS movie_keywords2_keyword_stats (mcv)  
ON k.keyword  
FROM movie_keywords2 mk JOIN keywords2 k ON (mk.keyword_id = k.id);  
ANALYZE movie_keywords2;  
  
-- Create cross-table MCV statistics on multiple filter columns (keyword + phonetic_code)  
CREATE STATISTICS movie_keywords2_multi_stats (mcv)  
ON k.keyword, k.phonetic_code  
FROM movie_keywords2 mk JOIN keywords2 k ON (mk.keyword_id = k.id);  
ANALYZE movie_keywords2;
```


Collect Join Statistics - Index-base Sampling

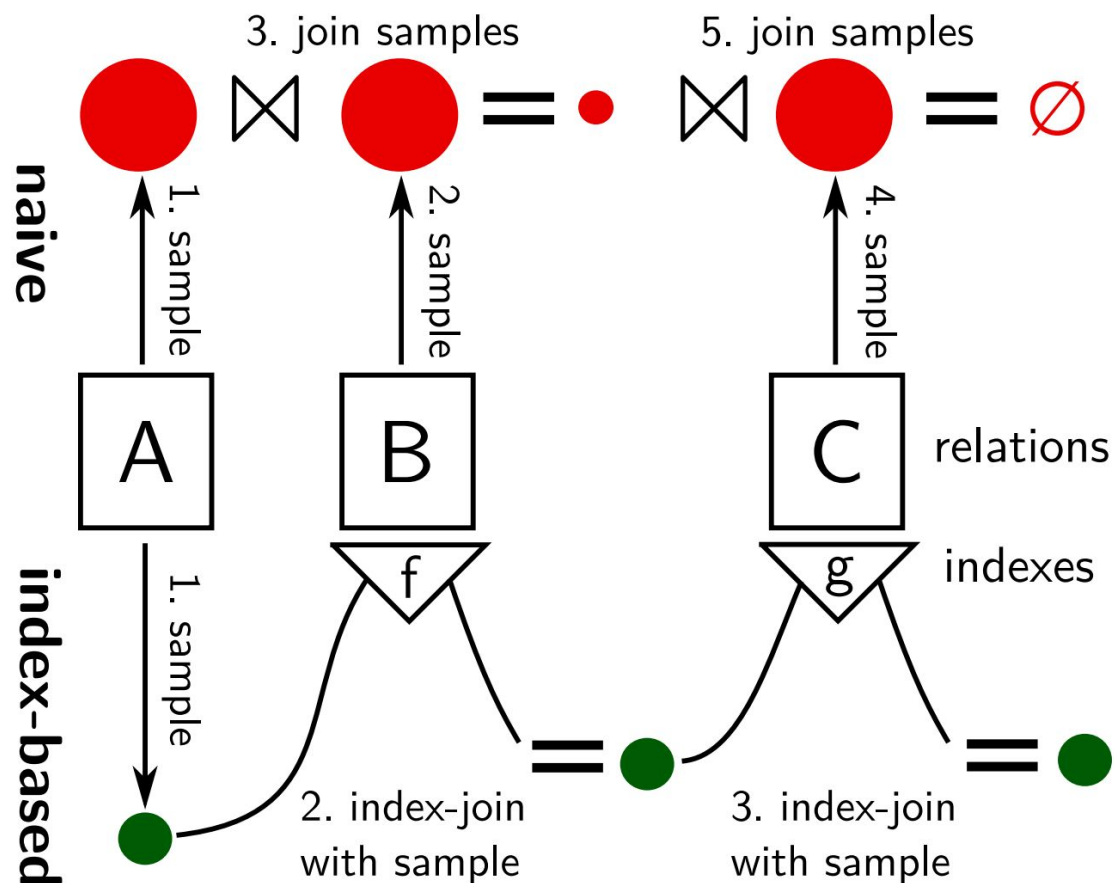


Figure 1: Naive (top) vs. index-based (bottom) join sampling

Cardinality Estimation Done Right: Index-Based Join Sampling

Viktor Leis, Bernhard Radke, Andrey Gubichev[†], Alfons Kemper, Thomas Neumann
 Technische Universität München
 {leis,radke,kemper,neumann}@in.tum.de
 Google[†]
 gubichev@google.com[†]

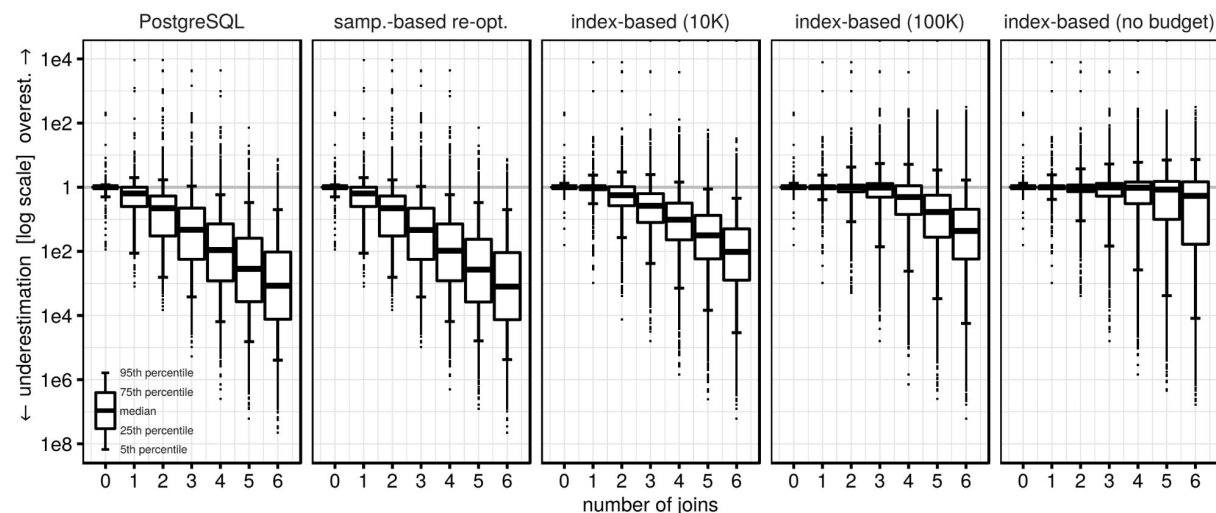


Figure 4: Quality of cardinality estimates for multi-join queries in comparison with the true cardinalities. Each boxplot summarizes the error distribution of all subexpressions with a particular size (over all queries in the workload)

WIP Patch for Join Statistics

<https://commitfest.postgresql.org/patch/6724/>

Performance

The Join Order Benchmark (JOB)

Setup:

- On my Macbook: ./configure 'CFLAGS=-g -O3'
- Cold runs: restarted database & cleared system cache
- Warm runs: repeat run of the entire test suite
- Average of 3 runs per test
- Add a single join MCV statistics object:

CREATE STATISTICS movie_keyword_keyword_stats (mcv)

ON k.keyword

FROM movie_keyword mk

JOIN keyword k ON (mk.keyword_id = k.id);

- **To avoid sampling noises, use the same database for the "with stats" vs "without stats" runs:**
 - CREATE STATISTICS + ANALYZE once, benchmark with stats, then DROP STATISTICS and benchmark without.

Cardinality Estimates

32 queries where the *keyword/movie_keyword* join appears in both plans with the same actual row count.

q-error = max(estimated/actual, actual/estimated). Lower is better.

	Without Stats	With Stats
Geometric mean	103.4x	4.2x
Median	131.7x	2.4x
90th percentile	1,230.6x	29.6x

- 16 improved, 0 regressed, 16 unchanged
- Unchanged queries have filters the MCV doesn't cover:
LIKE patterns, no filter on keyword, or rare values outside MCV

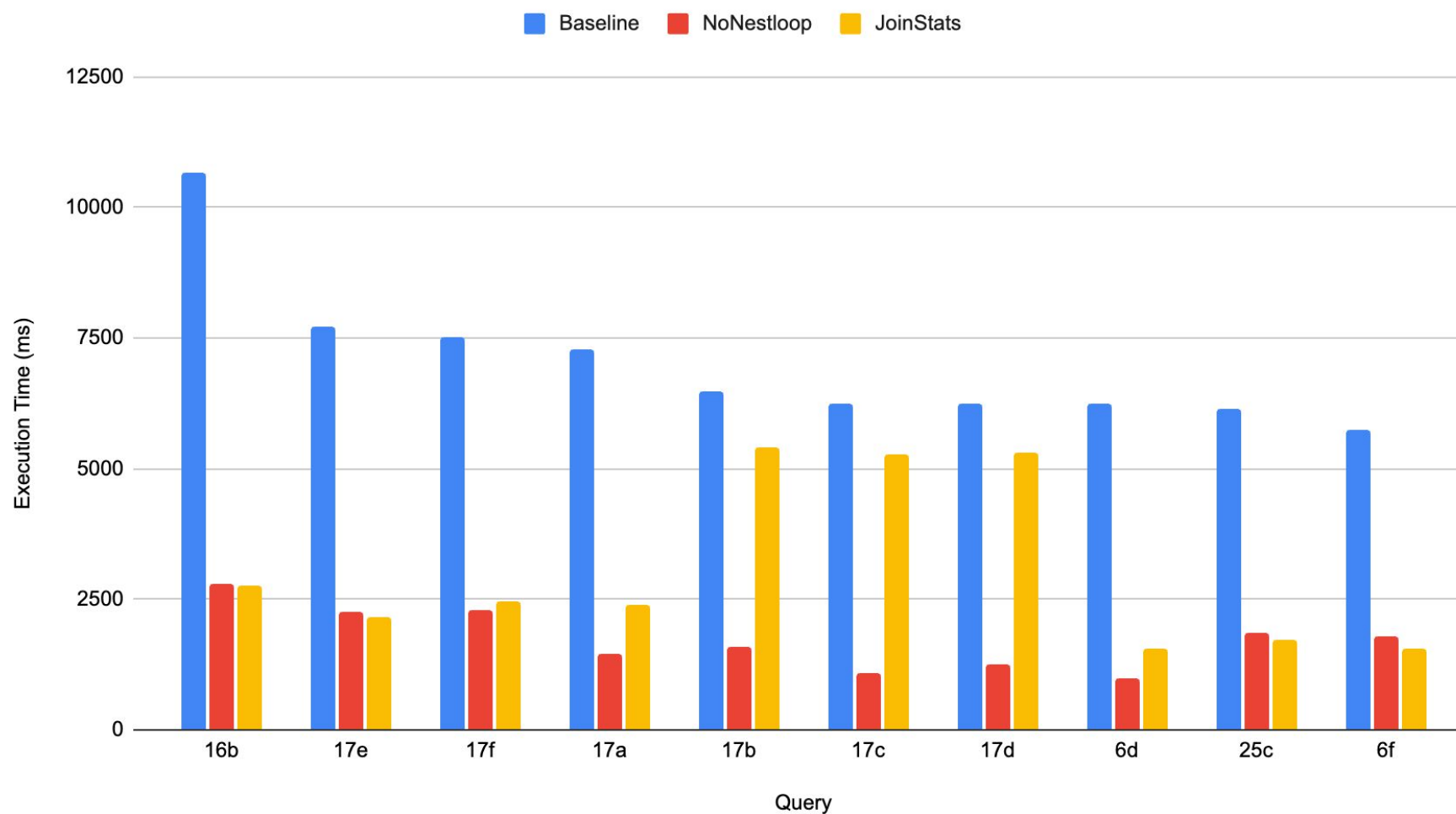
Total Execution Time

With a Single Join Statistic Object:

Configuration	Without Stats	With Stats	Speedup
Warm	114.1s	74.0s	1.54x
Cold	141.7s	103.7s	1.37x

Per-query Execution Times (from an older run)

Per-query Times for the 10 (out of 113) Slowest Queries



ANALYZE Costs

Configuration	ANALYZE time	Overhead
Baseline (No Join Stats)	266ms	-
1 join stat (mk -> (keyword, 135K rows))	487ms	+221ms (83%)

Alternatives and Open Questions

Three Independent Design Challenges for Join Statistics

Part 1: Syntax & Storage

How to **DEFINE** and **STORE**?

Part 2: Collection

How and when to **COLLECT**?

Part 3: Query Exploitation

How does the planner **FIND** and **USE** the right stat at plan time?

Syntax & Storage

Statistical Views (DB2 model)

```
-- SQL Server:
CREATE VIEW sales_product_v AS
  SELECT s.sale_id, s.amount, p.category, p.price
  FROM sales s JOIN products p ON (s.product_id = p.product_id)
  WHERE s.status = 'completed';

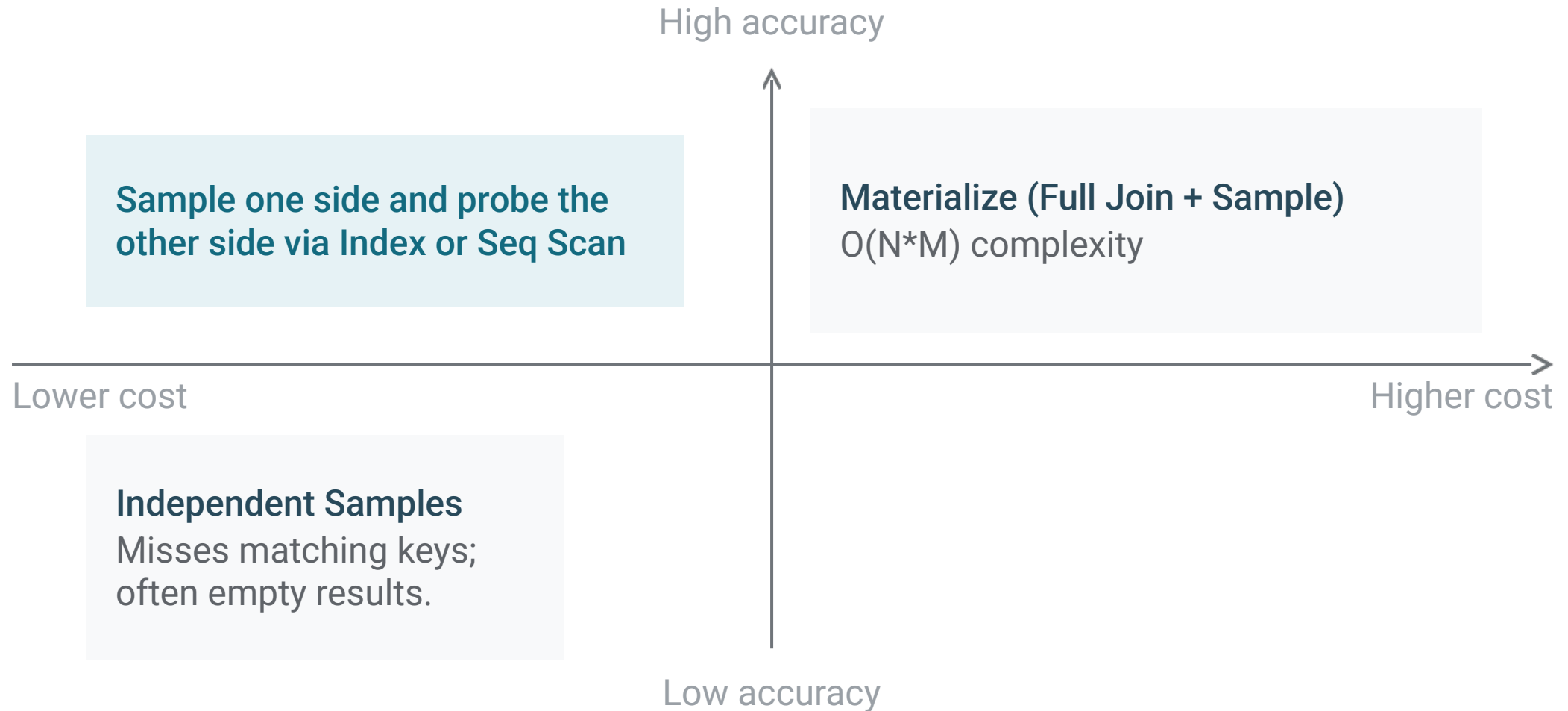
-- Mark the view as statistical view
ALTER VIEW sales_product_v ENABLE QUERY OPTIMIZATION;

-- Collect Statistics
RUNSTATS ON TABLE schema.sales_product_v WITH DISTRIBUTION;
```

DB2 Statistical Views

Statistics Collection

Ways to Collect Join Statistics



N-Way Joins (Not implemented yet)

Enumerate

- Stores intermediate results
- Results grow exponentially
- Set time or lookup budget

Don't Enumerate

- Use current query order
- Follow Constraint/Index graph

Query Exploitation

Subexpression Matching

Finding the right stat for a query is a restricted form of view matching.

W_q = query's full WHERE clause (including join clause)

W_v = stat's defining predicate

where W is decomposed into three logical parts:

$$W = PE \cap PR \cap PU$$

- Simpler if conservative syntactic matching only
- More complex if supports self-joins (NP-complete)
- More complex if supports Aggregates

Row Containment Tests:

Test	Description
Equijoin Subsumption (PE)	Every column equality in the statistic must also hold in the query. e.g., $R.x = S.y$
Range Subsumption (PR)	The stat's range predicates must be at least as wide as the query's. e.g., $R.price > 100$
Residual Subsumption (PU)	Every remaining conjunct in the stat matches one in the query. e.g., $R.name \text{ LIKE } \%abc\%$, $R.a + S.b > 10$

Deployed Expression Matching Solutions

System	Feature	Scope
SQL Server	<u>Index views</u>	Inner equi-joins, no self-joins
Snowflake	<u>Materialized views</u>	Single-table only
Redshift	<u>Materialized views</u>	Inner equi-joins, limited agg
DB2	<u>Statistical views</u>	Unclear

A good starting point: **Inner Equality Join**

What Else Can Benefit from Expression Matching

- Materialized View Rewrite
- Query Plan Caching
- Query Result Caching
- Training an AI model for Statistics Estimate or Query Plan?

Beyond Join Statistics

Learned & Feedback-Based Optimization

Learn Cardinality from Data

Train model -> predict row counts

Examples:

- MSCN (learn: query -> cardinality)
- DeepDB (learn: data distribution)
- NeuroCard (learn: full joint model generatively)
- ByteCard (in production)

PostgreSQL challenges:

No pluggable CE interface in planner; Training/feedback infrastructure required

Learn from Past Executions

Store execution stats -> improve future plans

Examples:

- SQL Server CE feedback
- Presto history-based optimization (HBO)

([Leis et al. CIDR 2019](#); [Hilprecht et al. PVLDB 2020](#); [Yang et al. PVLDB 2021](#), [Han et al., SIGMOD 2024](#))

([Microsoft](#); [PostgresPro](#); [Meta](#))

Runtime Adaptation

Correct mistakes **during execution**

Nested Loop → Hash Join

(after seeing 40k rows instead of 34)

Postgres Extension:

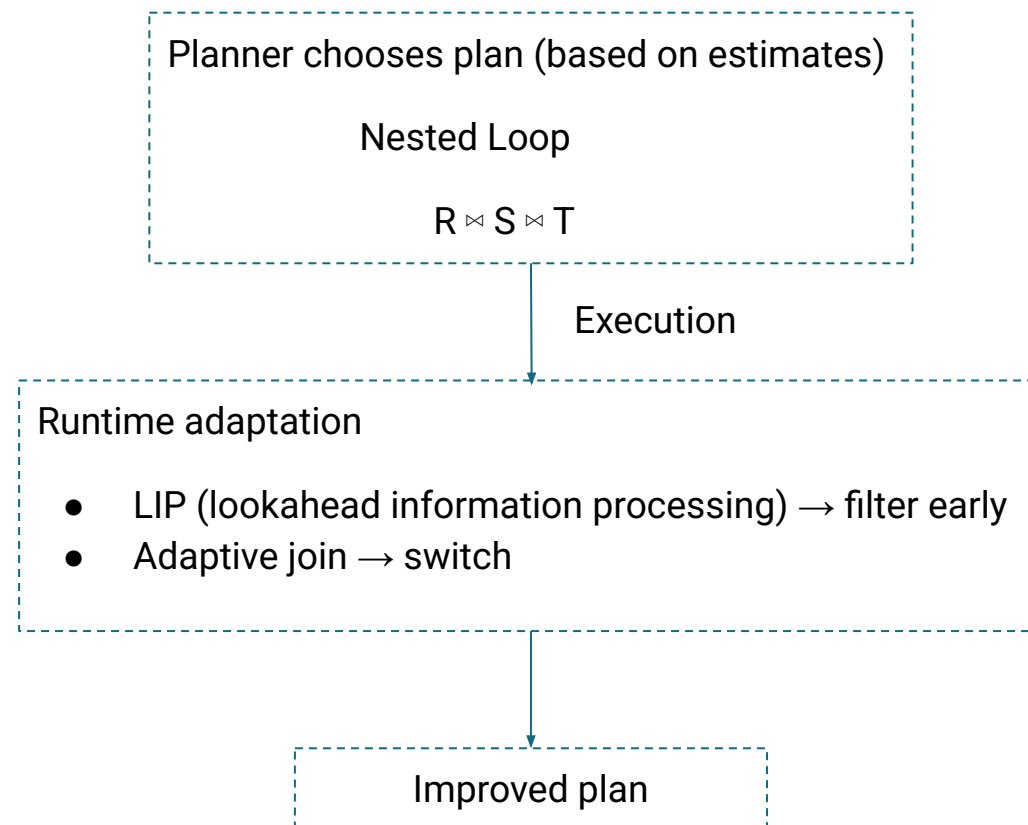
- Adaptive Query Optimization (AQO)

PostgreSQL: simple adaptive techniques:

- Early filtering (Bloom filters)
- Adaptive Join switching

≈ or > **learned optimizers on JOB workloads**

(Zhang et al., VLDB 2023)



Pessimistic Bounds

Underestimate -> nested loop over millions of rows → catastrophic

Upper bounds -> planner chooses a safer plan

Research:

- Safebound
- LpBound

Tested in PostgreSQL:

- 80% lower end-to-end runtimes on JOB
- Nearly as good as using the true cardinalities

Zhang et al., SIGMOD 2025

Possible Directions for PostgreSQL

- **Better estimation**
 - Join statistics
 - Learned cardinality estimation
- **Adapt during execution**
 - Runtime adaptation
- **Avoid worst plans**
 - `pg_plan_advice` (PG19)
 - `pg_hint_plan`
 - Pessimistic bounds

References

- Hacker's Discussion: [Is there value in having optimizer stats for joins/foreignkeys?](#)
- Comitfest Entry: [Add join MCV statistics for selectivity estimation](#)
- Join Order Benchmark Repo: [l-wang/join-order-benchmark](#)
- [Leis et al., "How Good Are Query Optimizers, Really?" PVLDB 2015](#)
- [Leis et al., "Cardinality Estimation Done Right: Index-Based Join Sampling" \(CIDR 2017\)](#)
- [Goldstein & Larson, "Optimizing queries using materialized views: a practical, scalable solution", SIGMOD, 2001](#)
- [DB2 Statistical Views](#)
- [SQL Server Index Views](#)
- [Snowflake Materialized Views](#)
- [Redshift Materialized Views](#)
- Past Talks:
 - [Corey Huinker's Talk on Join Statistics](#)
 - [Helping Planner Help You](#)

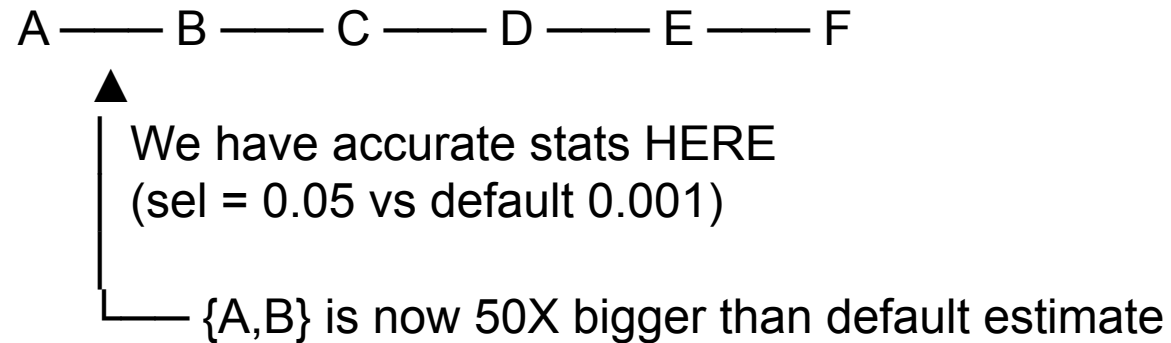
Thank You



Link to the resources and slides:
<https://github.com/l-wang/percona-live-2026>

“Island of Accuracy” - Accurate Locally, Worse Globally?

Consider a 6-way join query:



But then:

C joins {A,B}: C has even stronger correlation with A, but we didn't define that join stats

Will the more accurate selectivity for {A,B} help or hurt the final plan?